Pricing the American put via PSOR

Roberto Cepeda Rocandio

May 2026; revised October 2026

1 Setting and motivation

The American put can be exercised at any time $t \in [0, T]$, not only at maturity. This single change relative to the European case transforms the pricing problem qualitatively: the price now depends on a free *exercise boundary* $S_f(t)$ that separates a *continuation region* (where the option is held) from an *exercise region* (where it is exercised immediately). The boundary itself is unknown a priori — it is part of the solution.

The deliverable of this note is a numerical method that handles the free boundary cleanly, by reformulating the problem as a *Linear Complementarity Problem* (LCP) on a fixed domain and solving it iteratively at every time step using *Projected Successive Over-Relaxation* (PSOR). The implementation consists of two modules:

- `quantlib.psor`: a generic PSOR solver for tridiagonal LCPs (analogous to `quantlib.thomas`).
- `quantlib.cn_american`: the user-facing American put pricer, built on the θ-scheme infrastructure (`quantlib.theta_scheme`) with PSOR replacing Thomas in the per-step solve.

2 The American put as an optimal stopping problem

Under the risk-neutral measure $\mathbb{Q}$, the price of the American put is

$$V(S_0, 0) = \sup_{\tau \in \mathcal{T}_{[0,T]}} \mathbb{E}^{\mathbb{Q}}\big[\, e^{-r\tau}(K - S_\tau)^+ \,\big|\, S_0 \,\big], \tag{1}$$

where $\mathcal{T}_{[0,T]}$ is the set of stopping times taking values in $[0, T]$. Equation (1) states that the holder picks the stopping rule τ that maximises the discounted expected payoff.

The optimal τ partitions the state space (S, t) into two regions:

- **Continuation region** $\mathcal{C} = \{(S, t) : V(S, t) > (K - S)^+\}$: holding the option is strictly better than exercising now.
- **Exercise region** $\mathcal{E} = \{(S, t) : V(S, t) = (K - S)^+\}$: it is optimal (or at least no worse) to exercise.

The boundary between the two is the curve $S = S_f(t)$, the *exercise boundary*, also called the *free boundary* because its location is part of the solution rather than known a priori. For a put, $S_f(t)$ is increasing in t with $S_f(T^-) = K$.

3 The classical free-boundary PDE formulation

In the continuation region, the standard arbitrage argument shows that V satisfies the Black–Scholes PDE:

$$\frac{\partial V}{\partial t} + \tfrac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} - rV = 0, \qquad (S, t) \in \mathcal{C}. \tag{2}$$

In the exercise region, V equals the intrinsic value:

$$V(S,t) = K - S, \qquad (S,t) \in \mathcal{E}. \tag{3}$$

At the free boundary $S = S_f(t)$, two matching conditions hold:

$$V(S_f(t),t) = K - S_f(t), \tag{4}$$

$$\frac{\partial V}{\partial S}(S_f(t),t) = -1. \tag{5}$$

The first is value matching: V is continuous across the boundary. The second is the *smooth pasting* condition: V_S is also continuous. Two conditions are needed because the boundary itself is unknown; smooth pasting is the optimality condition that determines $S_f(t)$.

Why this formulation is hard numerically. The PDE (2) holds on the time-varying domain $\mathcal{C}$, whose boundary moves with t. The finite-difference methods for European options (FTCS, BTCS, CN) work on fixed domains; adapting the spatial mesh to track $S_f(t)$ would require remeshing at every time step, which is expensive and error-prone. We need a reformulation on a fixed domain.

4 The variational reformulation as an LCP

Switching to log-coordinates $x = \ln(S/K)$ and $\tau = T - t$ and denoting the spatial operator $\mathcal{L}V = \frac{\sigma^2}{2}V_{xx} + \mu V_x - rV$ as in the Crank–Nicolson note, we reformulate the American put as the system

$$\boxed{\begin{aligned} V_\tau - \mathcal{L}V &\geq 0 \\ V - g &\geq 0 \\ (V_\tau - \mathcal{L}V)(V-g) &= 0 \end{aligned}} \qquad \text{on } x \in (x_{\min}, x_{\max}),\ \tau \in (0,T], \tag{6}$$

where $g(x) = K(1-e^x)^+$ is the put payoff function (the *obstacle*). Initial condition: $V(x,0) = g(x)$. Boundary conditions: $V(x_{\min},\tau) = K - Ke^{x_{\min}}$ (deep ITM: exercise immediately, no discount) and $V(x_{\max},\tau) = 0$ (deep OTM: worthless).

Why each inequality is correct. Take them in turn.

1. $V_\tau - \mathcal{L}V \geq 0$. In $\mathcal{C}$ this is an equality (the BS PDE itself). In $\mathcal{E}$ we have $V \equiv g$, so $V_\tau = 0$, and a direct calculation $\mathcal{L}g = -rK \leq 0$ for $x < 0$ (the put-ITM region) makes $V_\tau - \mathcal{L}V = -\mathcal{L}g \geq 0$.

2. $V - g \geq 0$. The price never falls below the immediate payoff: if it did, exercising would dominate holding, an arbitrage.

3. $(V_\tau - \mathcal{L}V)(V-g) = 0$ *(complementarity)*. At every (x,τ), exactly one of the two preceding inequalities is strict: in $\mathcal{C}$ the second is strict ($V > g$) and the first is an equality (BS PDE); in $\mathcal{E}$ the first is strict ($V_\tau - \mathcal{L}V > 0$) and the second is an equality ($V = g$).

The structural payoff. The system (6) makes no reference to the free boundary S_f. The boundary emerges a posteriori as the locus where one inequality changes from strict to equality. Numerically, this is enormous: we can work on the fixed log-truncated domain of the finite-difference foundations note, with no need to track a moving boundary, and the complementarity condition automatically selects which nodes are in which region.

Equivalence. The LCP (6) is equivalent to the classical PDE+free-boundary formulation (2)–(5) for solutions $V \in C^1$. The smooth pasting condition is recovered automatically: from $V \geq g$ everywhere and $V = g$ on $\mathcal{E}$, taking the spatial derivative at the boundary gives $V_S \geq g_S = -1$ from the continuation side and $V_S = -1$ on the exercise side; combined with $V \in C^1$, this forces $V_S = -1$ at the boundary. Reference: Wilmott–Howison–Dewynne [1], Chapter 7.

5 Discretisation: from continuous to discrete LCP

Apply the θ-scheme to the operator $V_\tau - \mathcal{L}V$ exactly as in the Crank–Nicolson note. The discrete equality $AV^{n+1} = BV^n + \mathbf{b}_{\text{bnd}}$ (with A, B, $\mathbf{b}_{\text{bnd}}$ from that note) becomes the discrete LCP

$$\boxed{\begin{aligned} AV^{n+1} &\geq BV^n + \mathbf{b}_{\text{bnd}} \\ V^{n+1} &\geq \mathbf{g} \\ \left(AV^{n+1} - BV^n - \mathbf{b}_{\text{bnd}}\right) \cdot \left(V^{n+1} - \mathbf{g}\right) &= 0 \end{aligned}} \tag{7}$$

componentwise, where $\mathbf{g} \in \mathbb{R}^{N-1}$ is the payoff sampled on the interior grid: $g_j = K(1 - e^{x_j})^+$.

Each time step now requires solving the LCP (7) instead of a linear system. Thomas does not apply: it solves $Ax = b$, not $Ax \geq b$ with $x \geq c$. We need a new solver.

6 SOR: the iterative method behind PSOR

For the linear system $Ax = b$ with A tridiagonal and diagonal dominant, an alternative to Thomas is the iterative *Successive Over-Relaxation* (SOR) method. SOR produces a sequence $x^{(0)}, x^{(1)}, x^{(2)}, \ldots$ that converges to the exact solution. Although asymptotically more expensive than Thomas ($O(N \log \epsilon^{-1})$ per solve vs $O(N)$), SOR has a structural advantage: it generalises naturally to LCPs.

Definition. SOR updates one component at a time. For a tridiagonal system $a_{i,i-1}x_{i-1} + a_{ii}x_i + a_{i,i+1}x_{i+1} = b_i$, SOR uses

$$x_i^{(k+1)} = (1-\omega)\, x_i^{(k)} \;+\; \omega \cdot \frac{1}{a_{ii}} \Big(b_i \;-\; a_{i,i-1}\, x_{i-1}^{(k+1)} \;-\; a_{i,i+1}\, x_{i+1}^{(k)} \Big) \tag{8}$$

for $i = 1, \ldots, n$, with $\omega \in (0, 2)$ the *relaxation parameter*. At $\omega = 1$, equation (8) is plain Gauss–Seidel; at $\omega > 1$ it over-relaxes (extrapolates beyond the GS update); at $\omega < 1$ it under-relaxes (interpolates between old and new). Note that (8) uses $x_{i-1}^{(k+1)}$ (the just-updated value) and $x_{i+1}^{(k)}$ (the old value) — this in-place update is what makes SOR efficient.

Convergence. A classical result (Kahan; Ostrowski–Reich):

Theorem 6.1 (SOR convergence)**.** *If A is symmetric positive definite, SOR converges for every $\omega \in (0, 2)$ from any initial guess $x^{(0)}$. For a tridiagonal A with constant coefficients (our case), the convergence rate is geometric with optimal relaxation parameter*

$$\omega^* = \frac{2}{1 + \sqrt{1 - \rho_J^2}}, \tag{9}$$

where ρ_J is the spectral radius of the Jacobi iteration matrix $D^{-1}(L + U)$.

For the matrices arising from BTCS or CN, ρ_J is governed by the mesh ratio $\alpha = \sigma^2 \Delta\tau / (2\Delta x^2)$: ignoring the small convective asymmetry, $\rho_J \approx 2\theta\alpha/(1 + 2\theta\alpha) = 1 - O(1/\alpha)$. Refining the grid with $M \propto N$ makes $\alpha \propto N$, so $\omega^* \to 2$ like $2 - O(1/\sqrt{N})$. For the grids used here it stays well below 2: for Crank–Nicolson ($\theta = 1/2$), $\rho_J \approx 0.76$ and $\omega^* \approx 1.21$ at $N = 200$, $M = 100$, and $\rho_J \approx 0.86$, $\omega^* \approx 1.33$ on the 400×200 benchmark grid.

Asymmetry. The CN matrix A is not symmetric (the convective term μV_x uses centred differences, breaking symmetry). The Ostrowski–Reich theorem does not apply directly. However, A remains diagonally dominant, and a generalisation (Varga [8], Theorem 3.5) covers diagonally dominant M-matrices: SOR converges for $\omega \in (0, 2)$, with ω^* moderately shifted from (9). In practice the projected iteration has its optimum close to, and a little above, the linear prediction: $\omega = 1.2$ on the 100-unknown test problem of `validate_psor.py` (linear theory: 1.25), and about 1.5–1.6 on the 400×200 benchmark grid (linear theory: 1.33). The default $\omega = 1.2$ is a safe choice rather than the optimum; `benchmark_phase3.py` uses 1.4.

7 PSOR: SOR with projection

PSOR is the generalisation of SOR to linear complementarity problems. The modification is a single line: after each SOR component update, project onto the constraint $x_i \geq g_i$:

$$\boxed{x_i^{(k+1)} = \max\left\{ g_i,\ \ (1-\omega)\, x_i^{(k)} + \omega \cdot \frac{1}{a_{ii}}\Big(b_i - a_{i,i-1}\, x_{i-1}^{(k+1)} - a_{i,i+1}\, x_{i+1}^{(k)}\Big) \right\}.} \tag{10}$$

That is: compute the SOR candidate, and if it falls below the obstacle g_i, raise it to g_i. If it lies above, leave it.

Variational interpretation. The LCP (7) is equivalent (when A is symmetric positive definite) to the constrained minimisation

$$\min_{x \geq \mathbf{g}} \ \tfrac{1}{2} x^T A x - x^T b,$$

where $b = BV^n + \mathbf{b}_{\text{bnd}}$. PSOR is the componentwise application of SOR to the KKT conditions of this minimisation, and the projection $\max\{g_i, \cdot\}$ is exactly the projection onto the feasible set $\{x : x \geq \mathbf{g}\}$. This construction is due to Cryer [5].

Theorem 7.1 (Cryer 1971)**.** *If $A \in \mathbb{R}^{n \times n}$ is symmetric positive definite and $\omega \in (0, 2)$, the PSOR iteration* (10) *converges to the unique solution of the LCP $Ax \geq b$, $x \geq \mathbf{g}$, $(Ax - b)(x - \mathbf{g}) = 0$ from any initial guess $x^{(0)} \geq \mathbf{g}$.*

For non-symmetric A. The CN matrix is not symmetric, but is a strictly diagonally dominant M-matrix in the project's parameter range. PSOR convergence in this setting follows from results of Mangasarian and others; see Cottle–Pang–Stone [6], §5.3 and §5.6 for the rigorous treatment. The qualitative picture is that for moderate Pe'clet number, asymmetry slows convergence by a constant factor but does not destroy it.

Initialisation. A natural choice is $x^{(0)} = \mathbf{g}$ (the obstacle itself). It is feasible by construction, and is exact on the eventual exercise region. An alternative is warm-starting from the previous time step, $x^{(0)} = V^n$. Since $V^{n+1} - V^n = O(\Delta\tau)$, the warm start saves roughly half the iterations in practice.

8 Stopping criterion

The iteration must be terminated at finite k. The standard test is on the increment:

$$\left\| x^{(k+1)} - x^{(k)} \right\|_\infty < \texttt{tol}_{\text{abs}} + \texttt{tol}_{\text{rel}} \cdot \left\| x^{(k)} \right\|_\infty, \tag{11}$$

with typical values $\texttt{tol}_{\text{abs}} = 10^{-8}$ and $\texttt{tol}_{\text{rel}} = 10^{-7}$. The absolute component protects against deep-OTM puts where prices are small (cents); the relative component protects against deep-ITM puts where prices are large (close to K). The mixed criterion is robust across the entire strike range.

Why not the residual. The residual criterion $\|Ax - b\| <$ `tol`, standard for linear systems, does not apply to the LCP: the residual is non-zero in the exercise region (by the first inequality) even at the optimum. The increment criterion is agnostic to the problem type and works for both linear systems and LCPs.

Maximum iteration safeguard. If ω is mis-tuned or the matrix is poorly conditioned, the iteration may converge slowly or not at all. A hard limit `max_iter` (typically 10^4) prevents an infinite loop. Reaching the limit raises a `RuntimeError` (or `std::runtime_error` in C++), making a silent failure visible to the caller.

9 Computational cost

Per time step:

- Each PSOR sweep is $O(N)$ operations.
- Number of sweeps to reach tolerance ϵ: $K \sim -\log\epsilon / \log(1/\rho(\omega))$. At the optimum $\rho(\omega^*) = \omega^* - 1 = 1 - O(1/\sqrt{\alpha})$, so $K = O(\sqrt{\alpha}\,\log(1/\epsilon))$.
- Cost per time step: $O(NK)$.

With $M \sim N$ (matched to spatial accuracy as in CN), $\alpha \propto N$ and the total cost is $O(N^{5/2})$ up to the logarithm. Measured on the 400×200 benchmark grid with the default tolerances, PSOR needs about 30 sweeps per time step at $\omega = 1.2$ and 18 at $\omega = 1.5$, against one Thomas solve for the European pricer: a few tens of times more work per step, the price of LCP generality.

Comparison with other American pricers.

Method	Cost	Accuracy
CRR binomial (`quantlib.american`)	$O(N^2)$	$O(1/N)$, $O(1/N^2)$ Richardson
LSM (`quantlib.american`)	$O(N_{\text{paths}} N_{\text{steps}})$	MC, $O(1/\sqrt{N_{\text{paths}}})$
CN-PSOR (this note)	$O(N^3 \log \epsilon^{-1})$	$O(\Delta x^2 + \Delta\tau^2)$

CN-PSOR is the most accurate but most expensive at moderate N. LSM scales best to high dimensions; CRR is the simplest and most robust. The finite-difference/lattice benchmark (`benchmark_phase3.py`) makes the comparison numerical.

10 Implementation outline

Two modules:

`quantlib.psor` / `quant::pde::psor`. Generic LCP solver. The signature mirrors Thomas:

```
def psor_solve(sub, diag, sup, rhs, obstacle, *,
               omega=1.2, tol_abs=1e-10, tol_rel=1e-9,
               max_iter=10000, x0=None) -> (x, n_iter)
```

Returns both the solution and the number of iterations performed (the iteration count is diagnostic data; `validate_psor.py` uses it in its ω sweep).

`quantlib.cn_american` / `quant::pde::cn_american`. American put pricer. Reuses the θ-scheme grid and boundary construction of the Crank–Nicolson pricer, replacing the per-step Thomas solve with a PSOR solve. Two notable differences from `cn_european_put`:

1. The lower boundary condition is $V(x_{\min}, \tau) = K - Ke^{x_{\min}}$ (no discount, since exercise is immediate).

2. No Rannacher smoothing. Justification: the Rannacher protocol addresses the kink in the European payoff, which produces high-frequency oscillations that CN cannot damp. For the American put the oscillations are the same in nature, but they are *absorbed by the projection step*: nodes near the strike fall in the exercise region $\mathcal{E}$ where $V = g$ exactly, so the oscillation is clipped at g before it can propagate. Empirically, CN-PSOR for the American put has $O(\Delta\tau^2)$ convergence without Rannacher; the projection does the smoothing implicitly.

11 Validation strategy

Seven tests certify the new pricer:

1. **Cross-validation against CRR binomial** (`quantlib.american`): the gold-standard reference for American puts. Tolerance comparable to the discretisation error.

2. **Cross-validation against LSM** (`quantlib.american`): a secondary check. Tolerance loosened to account for Monte Carlo variance.

3. **Quadratic convergence in Δx**: doubling N should reduce the error by approximately 4. Demonstrates that the $\theta = 1/2$ scheme retains its second-order spatial accuracy under the projection.

4. **American $\geq$ European**: at every spot, the American put price must be at least the European put price. Strict in the exercise region, equality possible deep in the continuation region.

5. **ω-sweep convergence**: at fixed problem, $\omega \in \{0.5, 0.8, 1.0, 1.2, 1.4, 1.6, 1.8\}$ produces different iteration counts with an interior minimum. It occurs at $\omega = 1.2$, in line with the linear-theory value $\omega^* \approx 1.25$ for that test matrix.

6. **ω outside $(0, 2)$ raises**: $\omega = 0$ or $\omega = 2$ produces no-convergence within `max_iter`, raising a runtime error.

7. **Free boundary monotonicity**: the recovered exercise boundary $S_f(\tau)$ (the largest S such that $V = g$ at each τ) is monotonically increasing in τ, with $S_f(\tau \to T) \to K$. A qualitative shape test that catches gross errors.

References

References

[1] P. Wilmott, S. Howison and J. Dewynne. *The Mathematics of Financial Derivatives.* CUP, 1995. Chapter 7 on American options as free-boundary problems; self-contained derivation of smooth pasting from optimality.

[2] P. Wilmott. *Paul Wilmott on Quantitative Finance.* Wiley, 1998 (2nd ed. 2006). Chapters 8 and 35: practical PSOR implementation with code.

[3] D. Tavella and C. Randall. *Pricing Financial Instruments: The Finite Difference Method.* Wiley, 2000. Chapter 5 on American options; explicit treatment of PSOR with code.

[4] R. Seydel. *Tools for Computational Finance.* 6th ed., Springer, 2017. §4.5 on American options; concise.

[5] C. W. Cryer. *The solution of a quadratic programming problem using systematic overrelaxation.* SIAM Journal on Control 9, 385–392, 1971. Original PSOR paper: proof of convergence for symmetric positive-definite LCPs.

[6] R. W. Cottle, J.-S. Pang and R. E. Stone. *The Linear Complementarity Problem.* SIAM Classics in Applied Mathematics, 2009 (orig. Academic Press, 1992). The rigorous reference for LCP theory. §5.2–5.3 cover PSOR convergence in detail.

[7] R. Glowinski. *Numerical Methods for Nonlinear Variational Problems.* Springer, 2008 (orig. Springer 1984). The variational treatment of obstacle problems.

[8] R. S. Varga. *Matrix Iterative Analysis.* Revised 2nd ed., Springer, 2000. Chapters 3–4 on SOR convergence theory and optimal ω.

[9] Y. Saad. *Iterative Methods for Sparse Linear Systems.* 2nd ed., SIAM, 2003. Chapter 4 on relaxation methods, modern treatment.